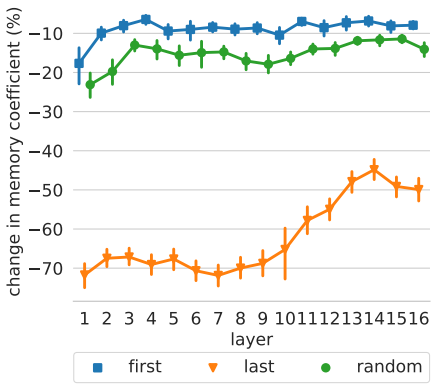

Figure 3: Relative change in memory coefficient caused by removing the first, the last, or a random token from the input.

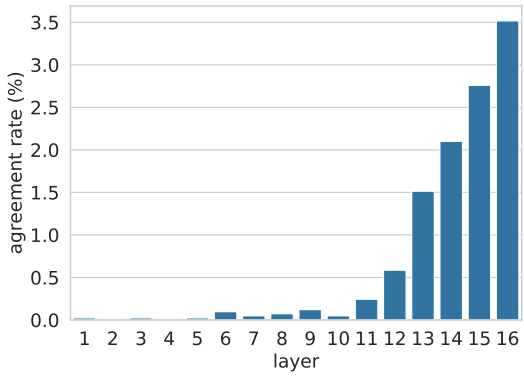

Figure 4: Agreement rate between the top-ranked token based on the value vector $\mathbf{v}_i^\ell$, and the next token of the top-ranked trigger example associated with the key vector $\mathbf{k}_i^\ell$.

key. Furthermore, the vast majority of retrieved prefixes (65%-80%) were associated with at least one identified pattern (Figure 2). Thus, the top examples triggering each key share clear patterns that humans can recognize.

Shallow layers detect shallow patterns Comparing the amount of prefixes associated with shallow patterns and semantic patterns (Figure 2), the lower layers (layers 1-9) are dominated by shallow patterns, often with prefixes that share the last word (e.g. $\mathbf{k}_{449}^1$ in Table 1). In contrast, the upper layers (layers 10-16) are characterized by more semantic patterns, with prefixes from similar contexts but without clear surface-form similarities (e.g. $\mathbf{k}_{1935}^{16}$ in Table 1). This observation corroborates recent findings that lower (upper) layers in deep contextualized models encode shallow (semantic) features of the inputs (Peters et al., 2018; Jawahar et al., 2019; Liu et al., 2019).

To further test this hypothesis, we sample 1600 random keys (100 keys per layer) and apply local modifications to the top-50 trigger examples of every key. Specifically, we remove either the *first*, *last*, or a *random* token from the input, and measure how this mutation affects the memory coefficient. Figure 3 shows that the model considers the end of an example as more salient than the beginning for predicting the next token. In upper layers, removing the last token has less impact, supporting our conclusion that upper-layer keys are less correlated with shallow patterns.

4 Values Represent Distributions

After establishing that keys capture patterns in training examples, we turn to analyze the information stored in their corresponding values. We show that each value $\mathbf{v}_i^\ell$ can be viewed as a distribution over the output vocabulary, and demonstrate that this distribution complements the patterns in the corresponding key $\mathbf{k}_i^\ell$ in the model's upper layers (see Figure 1).

Casting values as distributions over the vocabulary. We begin by converting each value vector $\mathbf{v}_i^\ell$ into a probability distribution over the vocabulary by multiplying it by the output embedding matrix E and applying a softmax:[4]

$$\mathbf{p}_i^\ell = \text{softmax}(\mathbf{v}_i^\ell \cdot E).$$

The probability distribution $\mathbf{p}_i^\ell$ is uncalibrated, since the value vector $\mathbf{v}_i^\ell$ is typically multiplied by the input-dependent memory coefficient $\mathbf{m}_i^\ell$, changing the skewness of the output distribution. That said, the *ranking* induced by $\mathbf{p}_i^\ell$ is invariant to the coefficient, and can still be examined. This conversion assumes (naïvely) that all model's layers operate in the same embedding space.

Value predictions follow key patterns in upper layers. For every layer ℓ and memory dimension i, we compare the top-ranked token according to $\mathbf{v}_i^\ell$, ($\text{argmax}(\mathbf{p}_i^\ell)$) to the next token w_i^ℓ in the top-1 trigger example according to $\mathbf{k}_i^\ell$ (the example whose memory coefficient for $\mathbf{k}_i^\ell$ is the highest). Figure 4 shows the *agreement rate*, i.e. the fraction of memory cells (dimensions) where the value's top prediction matches the key's top trigger example ($\text{argmax}(\mathbf{p}_i^\ell) = w_i^\ell$). It can be seen that the

[4]This is a simplification; in practice, we use the adaptive softmax (Baevski and Auli, 2019) to compute probabilities.

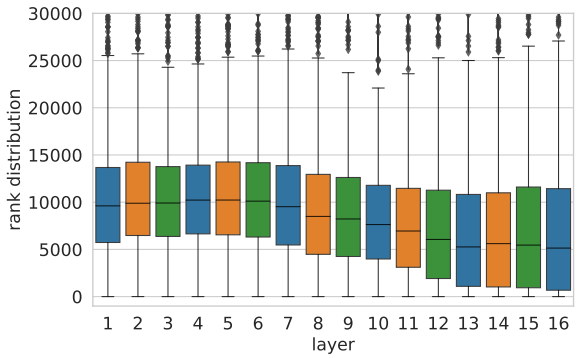

Figure 5: Distribution of the rank of the next-token in the top-1 trigger example of $\mathbf{k}_i^\ell$ (w_i^ℓ), according to the ranking induced by the value vector $\mathbf{v}_i^\ell$. We cut the tail of the distribution, which stretches up to the vocabulary size (∼270K tokens).

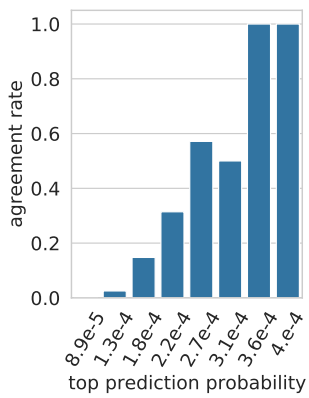

Figure 6: Agreement rate (between the top-ranked token based on the value vector $\mathbf{v}_i^\ell$ and the next token of the top-ranked trigger example associated with the key vector $\mathbf{k}_i^\ell$) as a function of the maximal probability assigned by the value vector.

agreement rate is close to zero in the lower layers (1-10), but starting from layer 11, the agreement rate quickly rises until 3.5%, showing higher agreement between keys and values on the identity of the top-ranked token. Importantly, this value is orders of magnitude higher than a random token prediction from the vocabulary, which would produce a far lower agreement rate (0.0004%), showing that upper-layer memories manifest non-trivial predictive power.

Next, we take the next token of $\mathbf{k}_i^\ell$'s top-1 trigger example (w_i^ℓ), and find where it ranks in the value vector's distribution $\mathbf{p}_i^\ell$. Figure 5 shows that the rank of the next token of a trigger example increases through the layers, meaning that w_i^ℓ tends to get higher probability in the upper layers.

Detecting predictive values. To examine if we can automatically detect values with high agreement rate, we analyze the probability of the values' top prediction, i.e., ($\max(\mathbf{p}_i^\ell)$). Figure 6 shows that although these distributions are not calibrated, distributions with higher maximum probabilities are more likely to agree with their key's top trigger example. We then take the 100 values with highest probability across all layers and dimensions (97 out of the 100 are in the upper layers, 11-16), and for each value $\mathbf{v}_i^\ell$, analyze the top-50 trigger examples of $\mathbf{k}_i^\ell$. We find that in almost half of the values (46 out of 100), there is at least one trigger example that agrees with the value's top prediction. Examples are provided in Table 2.

Discussion. When viewed as distributions over the output vocabulary, values in the upper layers tend to assign higher probability to the next-token of examples triggering the corresponding keys. This suggests that memory cells often store information on how to directly predict the output (the distribution of the next word) from the input (patterns in the prefix). Conversely, the lower layers do not exhibit such clear correlation between the keys' patterns and the corresponding values' distributions. A possible explanation is that the lower layers do not operate in the same embedding space, and therefore, projecting values onto the vocabulary using the output embeddings does not produce distributions that follow the trigger examples. However, our results imply that some intermediate layers *do* operate in the same or similar space to upper layers (exhibiting some agreement), which in itself is non-trivial. We leave further exploration of this phenomenon to future work.

5 Aggregating Memories

So far, our discussion has been about the function of a single memory cell in feed-forward layers. How does the information from *multiple* cells in multiple *layers* aggregate to form a model-wide prediction? We show that every feed-forward layer combines multiple memories to produce a distribution that is qualitatively different from each of its component memories' value distributions (Section 5.1). These layer-wise distributions are then combined via residual connections in a refinement process, where each feed-forward layer updates the residual's distribution to finally form the model's output (Section 5.2).

Value	Prediction	Precision@50	Trigger example
$\mathbf{v}^{15}_{222}$	*each*	68%	*But when bees and wasps resemble each*
$\mathbf{v}^{16}_{752}$	*played*	16%	*Her first role was in Vijay Lalwani's psychological thriller Karthik Calling Karthik, where Padukone was cast as the supportive girlfriend of a depressed man (played*
$\mathbf{v}^{13}_{2601}$	*extratropical*	4%	*Most of the winter precipitation is the result of synoptic scale, low pressure weather systems (large scale storms such as extratropical*
$\mathbf{v}^{15}_{881}$	*part*	92%	*Comet served only briefly with the fleet, owing in large part*
$\mathbf{v}^{16}_{2070}$	*line*	84%	*Sailing from Lorient in October 1805 with one ship of the line*
$\mathbf{v}^{12}_{3186}$	*jail*	4%	*On May 11, 2011, four days after scoring 6 touchdowns for the Slaughter, Grady was sentenced to twenty days in jail*

Table 2: Example values, their top prediction, the fraction of their key's top-50 trigger examples that agree with their prediction, and a matching trigger example (with the target token marked in blue).

5.1 Intra-Layer Memory Composition

The feed-forward layer's output can be defined as the sum of value vectors weighted by their memory coefficients, plus a bias term:

$$\mathbf{y}^\ell = \sum_i \text{ReLU}(\mathbf{x}^\ell \cdot \mathbf{k}^\ell_i) \cdot \mathbf{v}^\ell_i + \mathbf{b}^\ell.$$

If each value vector $\mathbf{v}^\ell_i$ contains information about the target token's distribution, how is this information aggregated into a single output distribution? To find out, we analyze the behavior of 4,000 randomly-sampled prefixes from the validation set. Here, the validation set is used (rather than the training set used to find trigger examples) since we are trying to characterize the model's behavior at inference time, not find the examples it "memorizes" during training.

We first measure the fraction of "active" memories (cells with a non-zero coefficient). Figure 7 shows that a typical example triggers hundreds of memories per layer (10%-50% of 4096 dimensions), but the majority of cells remain inactive. Interestingly, the number of active memories drops towards layer 10, which is the same layer in which semantic patterns become more prevalent than shallow patterns, according to expert annotations (see Section 3, Figure 2).

While there are cases where a single memory cell dominates the output of a layer, the majority of outputs are clearly compositional. We count the number of instances where the feed-forward layer's top prediction is *different* from all of the memories' top predictions. Formally, we denote:

$$\text{top}(\mathbf{h}) = \text{argmax}(\mathbf{h} \cdot E)$$

as a generic shorthand for the top prediction from the vocabulary distribution induced by the vector

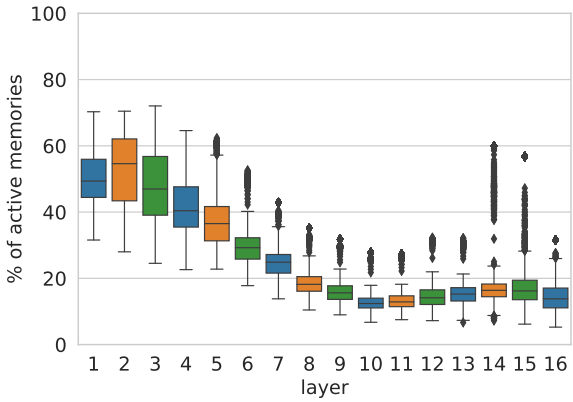

Figure 7: The fraction of active memories (i.e., with positive memory coefficient) out of 4096 memories in every layer, for a random sample of 4,000 examples.

$\mathbf{h}$, and compute the number of examples where the following condition holds:

$$\forall i : \text{top}(\mathbf{v}^\ell_i) \neq \text{top}(\mathbf{y}^\ell)$$

Figure 8 shows that, for any layer in the network, the layer's final prediction is different than *every one* of the memories' predictions in at least ∼68% of the examples. Even in the upper layers, where the memories' values are more correlated with the output space (Section 4), the layer-level prediction is typically not the result of a single dominant memory cell, but a composition of multiple memories.

We further analyze cases where at least one memory cell agrees with the layer's prediction, and find that (a) in 60% of the examples the target token is a common stop word in the vocabulary (e.g. *"the"* or *"of"*), and (b) in 43% of the cases the input prefix has less than 5 tokens. This suggests that very common patterns in the training data might